

DOCUMENTATION TECHNIQUE

DataShare

Plateforme de transfert sécurisé de fichiers

VERSION	DATE	MISSION
2.0 — MVP	Mai 2026	OpenClassrooms — P4

NestJS

Vue.js
3

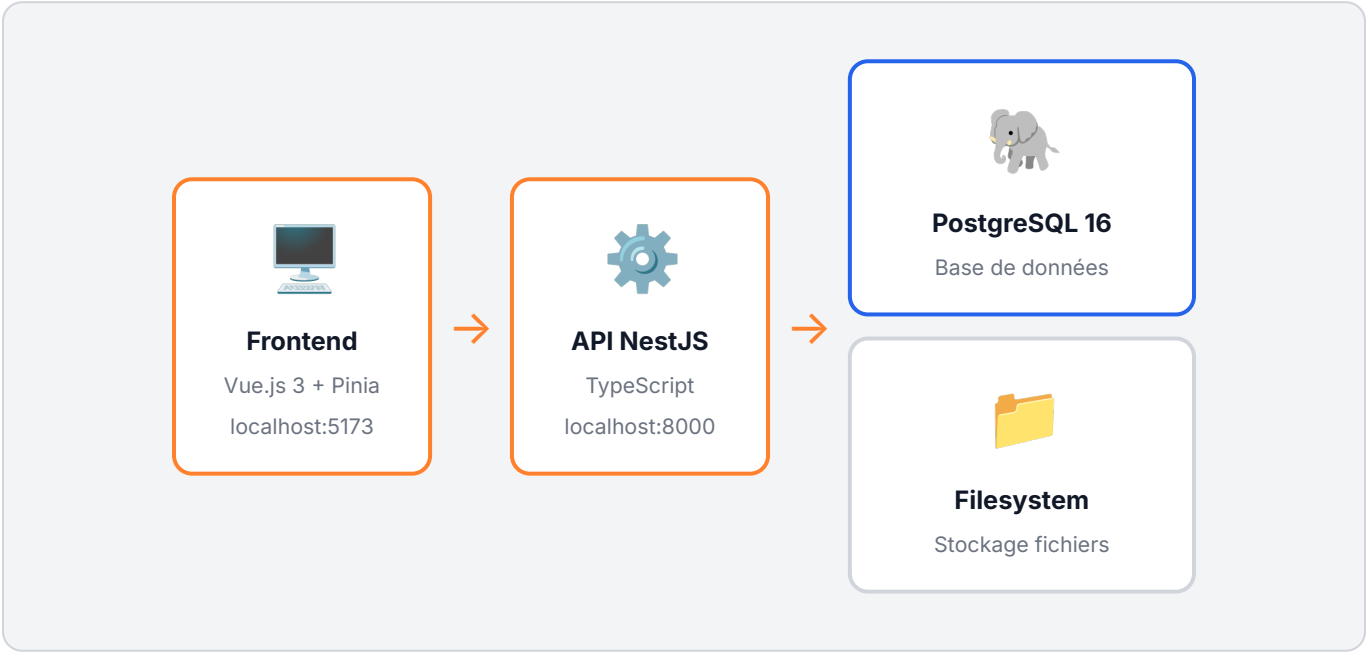
PostgreSQL

Docker

SECTION 01

Architecture de l'application

DataShare suit une **architecture 3-tiers** avec séparation stricte des responsabilités entre présentation, logique métier et persistance.



Flux d'une requête

- 1. Le **frontend Vue.js** envoie les requêtes via Axios avec le préfixe `/api`
- 2. Le **proxy Vite** redirige les appels vers NestJS sur le port 8000
- 3. **NestJS** valide le JWT (guard), valide les données (DTO), délègue au Service
- 4. Le **Service** interagit avec PostgreSQL via TypeORM et le disque via `fs`
- 5. La réponse JSON remonte en **snake_case** (compatibilité frontend)

Modules NestJS

Module	Responsabilité	Routes
AuthModule	Inscription, connexion, profil	/api/auth/*

FilesModule	Upload, download, historique, suppression, cron purge	/api/files/*
-------------	---	--------------

Déploiement : l'ensemble de l'application est conteneurisé avec Docker Compose — un seul `docker compose up -d` démarre la base de données, le backend et le frontend.

SECTION 02

Choix technologiques justifiés

● Backend — NestJS (TypeScript)

Architecture modulaire (Modules/Services/Controllers/Guards) imposant la séparation des responsabilités. TypeScript apporte le typage statique : erreurs détectées à la compilation. Décorateurs déclaratifs (`@Controller`, `@UseGuards`) lisibles et maintenables. Injection de dépendances intégrée → testabilité maximale par mock. Correspond aux exigences du cahier des charges.

● Frontend — Vue.js 3 + Pinia + Vite

Composition API pour une meilleure réutilisabilité. Pinia remplace Vuex avec une API plus simple et compatible TypeScript. Vite offre un démarrage serveur instantané et un HMR ultra-rapide en développement. Courbe d'apprentissage douce pour une équipe mixte.

● Base de données — PostgreSQL 16

Transactions ACID garanties. UUID natif et indexé — parfait pour les share_tokens non prédictibles. `timestamptz` inclut le fuseau horaire, évitant les bugs d'expiration. Contraintes `UNIQUE` et `CASCADE` assurées par la base, pas seulement l'application.

● Authentification — JWT

Stateless — pas de session côté serveur, simplifie le déploiement horizontal. Durée 7 jours pour le confort utilisateur sur un prototype. bcrypt coût 12 pour le hashage — résistant au brute-force. `passport-jwt` s'intègre nativement dans l'écosystème NestJS.

| Stockage — Système de fichiers local

Zéro dépendance externe pour un MVP. Chemin structuré `uploads/<userId>/<uuid>_<filename>` évite les collisions. Streaming avec `fs.createReadStream().pipe(res)` — pas de chargement en mémoire, critique pour les fichiers jusqu'à 1 Go. Migration vers AWS S3 prévue en production (la variable `UPLOAD_DIR` isole cette dépendance).

| Comparaison des alternatives écartées

Composant	Choix retenu	Alternative écartée	Raison
Backend	NestJS	Express.js	Trop permissif, pas de structure imposée
Backend	NestJS	Django	Non listé dans les contraintes techniques
Base de données	PostgreSQL	MongoDB	Relations entre entités → SQL plus adapté
State management	Pinia	Vuex	API plus simple, TypeScript natif

SECTION 03

Modèle de données

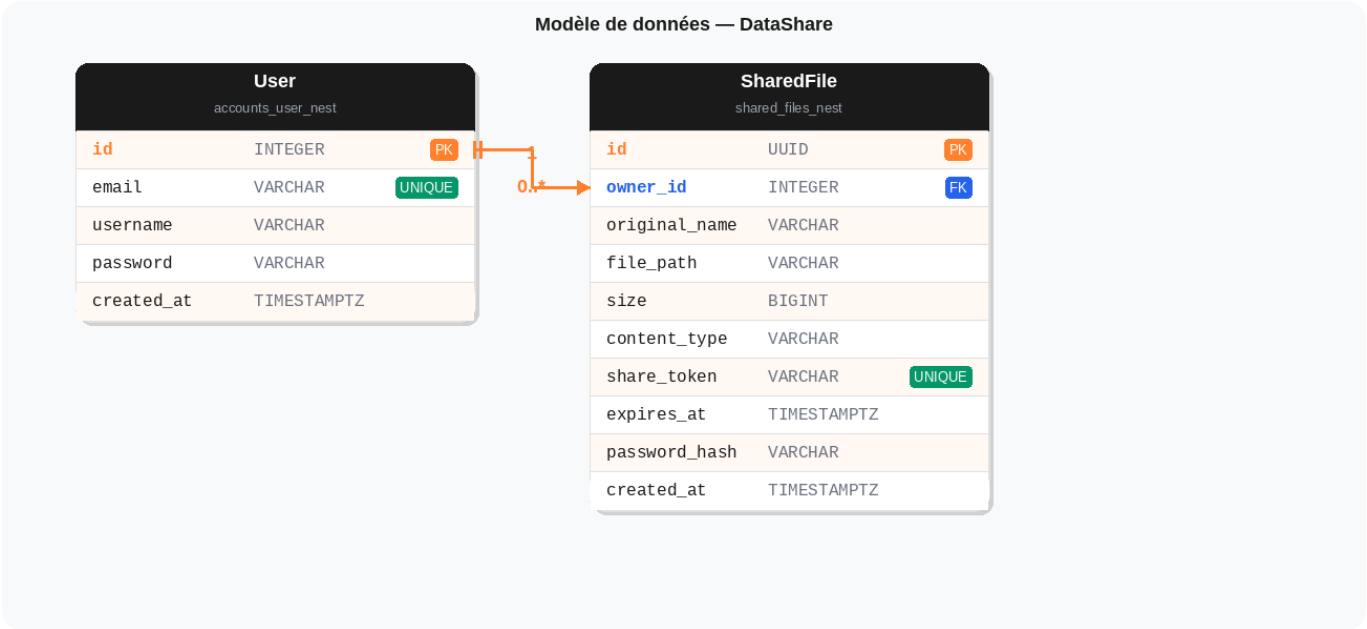


Table — User (accounts_user_nest)

Colonne	Type	Contrainte	Description
id	INTEGER	PK AUTO	Identifiant interne auto-incrémenté
email	VARCHAR	UNIQUE	Email de connexion — identifiant principal
username	VARCHAR	NOT NULL	Nom d'affichage
password	VARCHAR	NOT NULL	Hash bcrypt — jamais stocké en clair
created_at	TIMESTAMPZ	AUTO	Date d'inscription (auto)

Table — SharedFile (shared_files_nest)

Colonne	Type	Contrainte	Description
id	UUID	PK	UUID v4 — non prédictible
owner_id	INTEGER	FK NULL	Propriétaire (null = upload anonyme)
original_name	VARCHAR	NOT NULL	Nom du fichier pour Content-Disposition
file_path	VARCHAR	NOT NULL	Chemin absolu sur le disque
size	BIGINT	NOT NULL	Taille en octets (jusqu'à 1 Go)
content_type	VARCHAR	NOT NULL	Type MIME (ex: <code>application/pdf</code>)
share_token	VARCHAR	UNIQUE	UUID v4 — inclus dans le lien de partage
expires_at	TIMESTAMPTZ	NOT NULL	Date d'expiration du lien
password_hash	VARCHAR	DEFAULT ''	Hash bcrypt (vide = pas de protection)
created_at	TIMESTAMPTZ	AUTO	Date d'upload

SECTION 04

Documentation d'API

Base URL : `http://localhost:8000/api` | Auth : `Authorization: Bearer <JWT>`

Authentification

POST `/auth/register`

Public

Body : `{ email, username, password (min 8) }`

201 `{ id, email, username }` | 409 Email déjà utilisé | 400 Données invalides

POST `/auth/login`

Public

Body : `{ email, password }`

200 `{ access_token, user: { id, email, username } }` | 401 Identifiants invalides

GET `/auth/me`

 JWT requis

200 `{ id, email, username }` | 401 Token invalide

Fichiers

GET `/files`

 JWT requis

Historique des fichiers de l'utilisateur connecté.

200 Array de fichiers avec `share_url`, `is_expired`, `is_password_protected`

POST `/files/upload`

 JWT requis

multipart/form-data : `file` (requis), `password` (optionnel, min 6), `expiry_hours` (1-168)

201 Fichier créé | 403 Extension interdite | 413 Fichier > 1 Go

POST `/files/upload/anonymous`

Public

Même paramètres que l'upload connecté. Le fichier n'est lié à aucun compte (owner = null).

DELETE	/files/:id	 JWT requis
Suppression physique (disque) + métadonnées (base). 200 OK 404 Fichier inexistant ou appartenant à un autre utilisateur		
GET	/files/share/:token	Public
Métadonnées publiques : <code>original_name</code> , <code>size</code> , <code>expires_at</code> , <code>is_expired</code> , <code>is_password_protected</code> 200 OK 404 Token inconnu		
GET	/files/download/:token?password=xxx	Public
Streaming binaire du fichier. 200 OK 403 Mot de passe incorrect 410 Lien expiré 404 Token inconnu		

SECTION 05

Sécurité et gestion des accès

● JWT — Authentification stateless

Tokens HMAC-SHA256, durée 7 jours. Clé secrète en variable d'environnement. `passport-jwt` valide chaque requête protégée. Payload minimal : `{ sub, email }`.

● bcrypt — Hashage des mots de passe

Coût 12 itérations. Appliqué aux mots de passe utilisateurs ET aux mots de passe de fichiers. Jamais retourné dans les réponses API. Message d'erreur identique (email inconnu = mauvais MDP).

● Isolation des données

Chaque requête filtre par `ownerId = req.user.id`. La suppression retourne 404 (pas 403) si le fichier appartient à un autre — ne révèle pas l'existence.

● Validation des entrées

`class-validator` sur tous les DTOs. `ValidationPipe` global avec `whitelist: true`. Vérifications serveur : taille max 1 Go, extensions interdites (`.exe`, `.bat`...), durée 1-168h.

| Scan de sécurité des dépendances

```
$ npm audit  
  
found 0 vulnerabilities
```

Résultat : aucune vulnérabilité CVE détectée dans les dépendances de production NestJS (Mai 2026).

| Tokens de partage

Les liens de partage utilisent des **UUID v4** (128 bits d'entropie). La probabilité de deviner un token valide est de 1 sur 2^{122} , soit environ 1 sur 5×10^{36} — pratiquement impossible.

Améliorations prévues pour la production

Amélioration	Priorité	Raison
Cookies <code>httpOnly</code> (remplacement localStorage)	Haute	Protection XSS
HTTPS / TLS	Haute	Chiffrement en transit
Rate limiting sur <code>/api/auth/*</code>	Haute	Protection brute-force
Migration fichiers vers AWS S3	Moyenne	Scalabilité
Scan antivirus uploads (ClamAV)	Moyenne	Sécurité malware

SECTION 06

Qualité, tests et maintenance

18

Tests unitaires

18/18

Tests passants

~87%

Couverture services

0

Vulnérabilités npm

Tests unitaires — Jest

Fichier	Tests	US couvertes	Statements
<code>auth.service.spec.ts</code>	7	US03, US04	86.6%
<code>files.service.spec.ts</code>	11	US01, US02, US05, US06, US09	89%

Stratégie de mock

Dépendance	Stratégie	Raison
Repository TypeORM	Mock complet (<code>jest.fn()</code>)	Pas de vraie base de données
<code>fs</code> (filesystem)	Mock partiel (<code>jest.requireActual</code>)	TypeORM utilise fs en interne
<code>bcrypt</code>	Mock complet	Tester les appels, pas l'algorithme
<code>uuid</code>	Mock (valeur fixe)	Assertions prédictibles

Exécution des tests

```
cd backend-nest
npm test          # 18 tests unitaires
npm run test:cov  # Rapport de couverture HTML
```

Maintenance des dépendances

Type	Fréquence	Commande
Correctifs de sécurité	Immédiat	<code>npm audit fix</code>
Mises à jour mineures	Mensuelle	<code>npm update</code>
Mises à jour majeures	Trimestrielle	Évaluation + branche dédiée

SECTION 07

Processus d'installation et d'exécution

Prérequis

● Docker (recommandé)

Docker ≥ 24 & Docker Compose ≥ 2.20
Un seul commande pour tout lancer.

● Développement local

Node.js ≥ 20
PostgreSQL 16 (ou via Docker)

Installation rapide (Docker)

```
git clone https://github.com/maliciouuus/P4_architect
cd P4_architect
docker compose up -d
# Frontend : http://localhost:5173
# API      : http://localhost:8000/api
```

Installation développement local

```
# 1. Base de données
docker compose up -d db

# 2. Backend NestJS
cd backend-nest
npm install
npm run start:dev      # http://localhost:8000

# 3. Frontend Vue.js
cd ../frontend
npm install
npm run dev            # http://localhost:5173
```

Variables d'environnement

Variable	Défaut	Description
<code>DATABASE_URL</code>	<code>postgresql://datashare:datashare@...</code>	Connexion PostgreSQL
<code>JWT_SECRET</code>	<code>change_me_in_production</code>	À changer en production
<code>MAX_FILE_SIZE</code>	<code>1073741824</code>	Taille max en octets (1 Go)
<code>SHARE_LINK_EXPIRY_HOURS</code>	<code>168</code>	Durée de validité (7 jours)
<code>UPLOAD_DIR</code>	<code>uploads</code>	Dossier de stockage
<code>FRONTEND_URL</code>	<code>http://localhost:5173</code>	CORS + liens de partage

Configuration base de données

```
-- scripts/setup_db.sql
CREATE USER datashare WITH PASSWORD 'datashare';
CREATE DATABASE datashare OWNER datashare;
GRANT ALL PRIVILEGES ON DATABASE datashare TO datashare;
```

SECTION 08

Utilisation de l'IA dans le développement

L'IA (Claude — Anthropic) a été utilisée comme **outil de support technique** sur des tâches périphériques au développement : génération de tests, aide sur des commandes complexes, débogage et documentation. Le code métier de l'application (backend NestJS, frontend Vue.js, logique d'authentification, gestion des fichiers) a été entièrement écrit par le développeur.

Tâches confiées à l'IA

Domaine	Tâche confiée	Rôle du développeur
Tests unitaires	Génération des squelettes de tests Jest pour les controllers et la stratégie JWT (fichiers <code>*.controller.spec.ts</code> , <code>jwt.strategy.spec.ts</code>)	Définition de la stratégie de mock, correction des problèmes d'import natif (bcrypt/typeorm), validation que les assertions testent bien le comportement attendu
Commandes terminal	Aide sur des commandes complexes : configuration Docker Compose, génération du rapport de couverture Jest, commandes TypeORM pour les migrations, scripts Python pour les PDFs	Adaptation au contexte du projet, vérification que les commandes correspondent à la stack utilisée (Node 16, NestJS 11)
Débogage	Identification de la cause des erreurs TypeScript à la compilation, incompatibilités entre modules NestJS lors des tests	Compréhension de la cause racine, décision de la correction à appliquer
Documentation	Aide à la rédaction des fichiers TESTING.md, SECURITY.md, PERF.md, MAINTENANCE.md et de cette documentation technique	Définition du contenu, révision de chaque section, ajustement selon les exigences du cahier des charges

Ce qui n'a pas été délégué à l'IA

- **Architecture de l'application** — choix NestJS/Vue/PostgreSQL, découpage en modules, conception des entités
- **Logique métier** — services d'authentification, gestion des fichiers, token de partage, expiration automatique
- **Sécurité** — décisions bcrypt coût 12, JWT, CORS, validation DTOs, contrôle d'accès par ownerId
- **Interface utilisateur** — toutes les vues Vue.js, le routeur, les stores Pinia

Supervision appliquée

Chaque suggestion de l'IA a été relue et validée avant d'être intégrée. Par exemple, les tests générés utilisaient initialement des imports directs de `bcrypt` qui échouaient à cause du module natif node-gyp — le développeur a identifié le problème et décidé de la stratégie de mock correcte (`jest.mock('bcrypt')` + `jest.mock('typeorm')`).

Bilan

L'IA a accéléré les tâches répétitives ou techniques (tests, commandes, documentation) sans se substituer au raisonnement architectural. Le développeur est resté décisionnaire sur chaque choix structurant du projet.

9

Endpoints API

32

Tests unitaires

78%

Couverture

0

Vulnérabilités